Grade Calculator Example in TypeScript

TypeScript + HTML

Install TypeScript

Use npm to install typescript.

```
npm install -g typescript
```

Compile TypeScript to JavaScript

We need to compile TypeScript code using the `tsc` compiler.

Use `--project` so that `tsc` reads `tsconfig.json` (including `strict`). If you pass a file name such as `tsc src/app.ts`, `tsconfig.json` is ignored.

```
tsc --project src/tsconfig.json
```

tsconfig.json

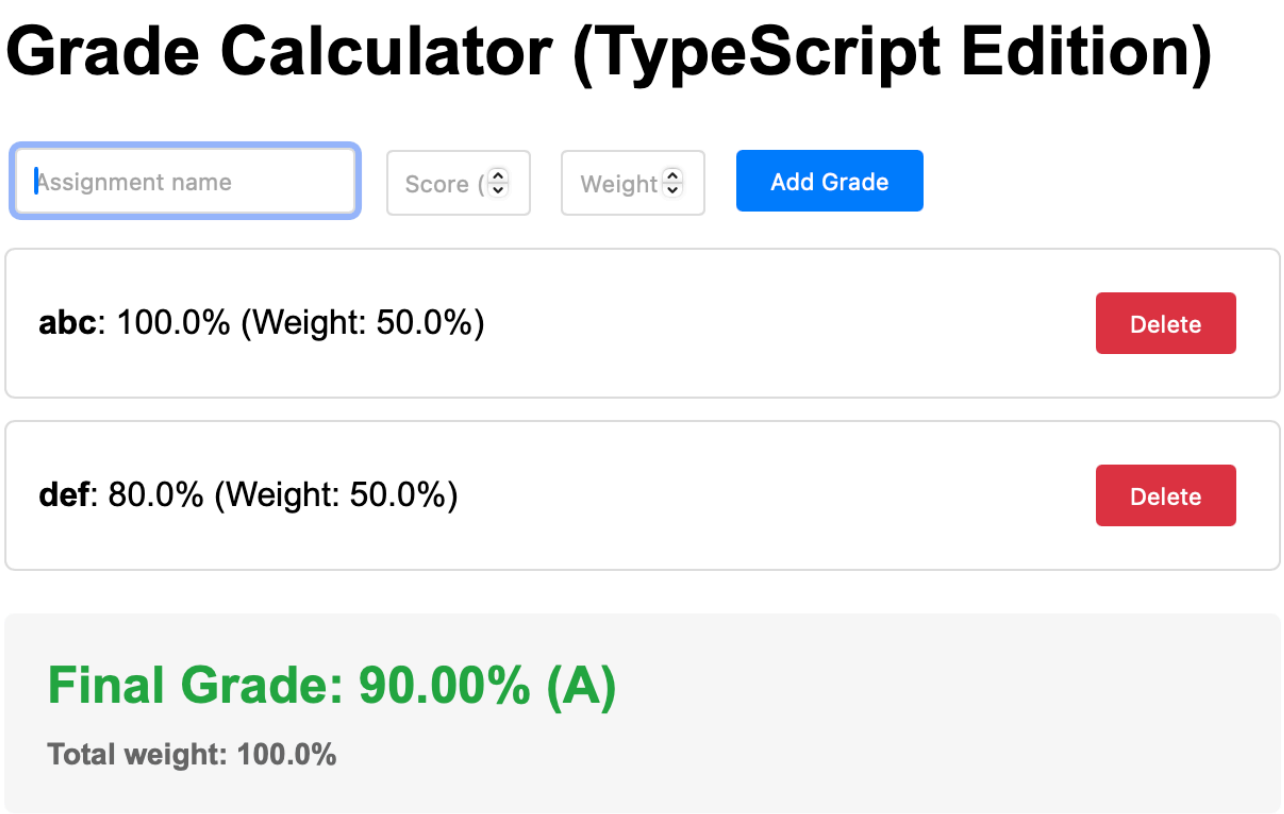
`tsconfig.json` has all the compiler configuration.

```
{
  "compilerOptions": {
    "target": "ES2018",
    "module": "es2015",
    "lib": ["ES2018", "DOM"],
    "outDir": "..",
    "rootDir": "./",
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true
  },
```

- The `noUnused*` and `noImplicitReturns` options turn sloppy code into compile errors.
- We can create a `tsconfig.json` using `tsc --init`, then adjust it.
- We run `tsc --project src/tsconfig.json` to compile the TypeScript code.

We can use watch mode `tsc --watch --project src/tsconfig.json` to automatically recompile on save.

HTML


Grade Calculator (TypeScript Edition)
Assignment name
Score (
Weight
Add Grade
abc: 100.0% (Weight: 50.0%)
Delete
def: 80.0% (Weight: 50.0%)
Delete
Final Grade: 90.00% (A)
Total weight: 100.0%

Almost the same as the JavaScript version, with two differences.

- The generated JavaScript is loaded from `../app.js` (`outDir` is `..`).
- There is **no** `onclick="addGrade()"` any more. `addGrade` is a `private` method of a class, so HTML cannot call it. The class registers the listener itself.

```
<div id="grade-form">
  <input type="text" id="assignment-name" placeholder="Assignment name">
  <input type="number" id="score" placeholder="Score (0-100)" min="0" max="100" step="0.1">
  <input type="number" id="weight" placeholder="Weight %" min="0" max="100" step="0.1">
  <button id="add-grade-btn" type="button">Add Grade</button>
</div>

<div id="error-message" class="error"></div>
<div id="grades-list"></div>
<div id="result" class="result"></div>

<script src="../app.js"></script>
```

Data Model: Types

```typescript
interface Grade {
  readonly id: number;   // can never change after creation
  name: string;
  score: number;
  weight: number;
}

interface GradeSummary {
  percentage: number;
  totalWeight: number;
}

// Only these five values are allowed
type LetterGrade = 'A' | 'B' | 'C' | 'D' | 'F';
```

Types disappear after compilation

Compare `src/app.ts` with the generated `app.js`.

- `interface Grade`, `interface GradeSummary`, and `type LetterGrade` **do not exist** in `app.js`.
- Types exist only for the compiler. At runtime, it is plain JavaScript.

This is why we also need runtime checks such as `instanceof` for DOM elements (see below).

Constants

```
const MAX_WEIGHT = 100;
const WEIGHT_EPSILON = 1e-9;
```

Why do we need `WEIGHT_EPSILON`? Decimal numbers are not exact in floating point:

```
console.log(0.1 + 0.2);          // 0.30000000000000004
console.log(0.1 + 0.2 === 0.3);  // false
```

If a user enters weights `33.3 + 33.3 + 33.4`, the sum may be slightly above `100`. We allow a tiny error when we compare.

Record: one color for every letter grade

```
const GRADE_COLORS: Record<LetterGrade, string> = {
  A: '#28a745',
  B: '#20c997',
  C: '#ffc107',
  D: '#fd7e14',
  F: '#dc3545'
};
```

- If we forget a key (for example `F`), the compiler reports an error.
- If we write `GRADE_COLORS['E']`, the compiler reports an error.
- No fallback such as `|| '#000'` is needed.

Pure Functions

These functions never touch the DOM, so we can test them without a browser.

```
function getLetterGrade(percentage: number): LetterGrade {
  if (percentage >= 90) return 'A';
  if (percentage >= 80) return 'B';
  if (percentage >= 70) return 'C';
  if (percentage >= 60) return 'D';
  return 'F';
}
```

Weighted average

```
function calculateFinalGrade(grades: Grade[]): GradeSummary | null {
  const totalWeight = grades.reduce((sum, grade) => sum + grade.weight, 0);
  if (totalWeight === 0) return null;

  const weightedSum = grades.reduce(
    (sum, grade) => sum + grade.score * grade.weight, 0);
  return { percentage: weightedSum / totalWeight, totalWeight };
}
```

With 100 (50%) and 80 (50%):

`(100 x 50 + 80 x 50) / (50 + 50) = 90` → **A**

The return type forces us to handle `null`

The return type is `GradeSummary | null`.

```
const summary = calculateFinalGrade(grades);

// summary.percentage;   // Error: 'summary' is possibly 'null'.

if (summary === null) {
  console.log('Total weight must be greater than 0');
} else {
  console.log(summary.percentage);   // OK: TypeScript knows it is not null
}
```

This is `strictNullChecks` (part of `strict`) protecting us from "forgot to handle the empty case" bugs.

Finding DOM Elements Safely

`document.getElementById` returns `HTMLElement | null`. We need a specific type such as `HTMLInputElement` to read `.value`.

```
function getElement<T extends HTMLElement>(
  id: string, type: new () => T): T {
  const element = document.getElementById(id);
  if (!(element instanceof type)) {
    throw new Error(`#${id} was not found or is not a ${type.name}`);
  }
  return element;
}

const input = getElement('score', HTMLInputElement);  // type: HTMLInputElement
```

- `instanceof` is a **real runtime check**. If the element is missing or has the wrong type, we get a clear error immediately.
- `as HTMLInputElement` is only an assertion. It does not check anything at runtime.
- This is a practical use of a generic function: one helper works for every element type.

Grade Calculator Class

The class holds the data, finds the UI elements once, and updates the screen.

1. Data and UI elements

```
class GradeCalculator {
  private grades: Grade[] = [];
  private nextId = 1;

  private readonly nameInput = getElement('assignment-name', HTMLInputElement);
  private readonly scoreInput = getElement('score', HTMLInputElement);
  private readonly weightInput = getElement('weight', HTMLInputElement);
  private readonly errorDiv = getElement('error-message', HTMLDivElement);
  private readonly gradesList = getElement('grades-list', HTMLDivElement);
  private readonly resultDiv = getElement('result', HTMLDivElement);
```

2. Constructor: event listeners

```
  constructor() {
    getElement('add-grade-btn', HTMLButtonElement)
      .addEventListener('click', () => this.addGrade());

    // Enter key support
    document.querySelectorAll('input').forEach(input => {
      input.addEventListener('keydown', (e) => {
        if (e.key === 'Enter') this.addGrade();
      });
    });
```

TypeScript **infers** that `e` is a `KeyboardEvent`, so `e.key` needs no cast.

2. Constructor: Delete buttons

The Delete buttons are created dynamically, so one listener on the list handles all of them (event delegation):

```
    this.gradesList.addEventListener('click', (event) => {
      if (!(event.target instanceof Element)) return;
      const button =
        event.target.closest<HTMLButtonElement>('[data-grade-id]');
      if (button) this.deleteGrade(Number(button.dataset.gradeId));
    });

    // Initial display
    this.render();
  }
```

3. addGrade: get values and validate

```
  private addGrade(): void {
    const name = this.nameInput.value.trim();
    const score = parseFloat(this.scoreInput.value);
    const weight = parseFloat(this.weightInput.value);

    this.errorDiv.textContent = '';   // Clear previous error

    if (!name) {
      this.errorDiv.textContent = 'Please enter an assignment name';
      return;
    }
    if (isNaN(score) || score < 0 || score > 100) {
      this.errorDiv.textContent = 'Score must be between 0 and 100';
      return;
    }
```

```
    if (isNaN(weight) || weight < 0 || weight > 100) {
      this.errorDiv.textContent = 'Weight must be between 0 and 100';
      return;
    }

    const currentWeight =
      this.grades.reduce((sum, grade) => sum + grade.weight, 0);
    if (currentWeight + weight > MAX_WEIGHT + WEIGHT_EPSILON) {
      this.errorDiv.textContent = 'Total weight cannot exceed 100%';
      return;
    }
```

3. addGrade: add and redraw

```
    const grade: Grade = { id: this.nextId++, name, score, weight };
    this.grades.push(grade);

    // Clear inputs
    this.nameInput.value = '';
    this.scoreInput.value = '';
    this.weightInput.value = '';
    this.nameInput.focus();

    this.render();   // Update the whole screen
  }
```

The `id` comes from a counter (`nextId`), so two grades never get the same id.

4. delete and render

Every change calls `render()` once. We cannot forget to update half of the screen.

```
  private deleteGrade(id: number): void {
    this.grades = this.grades.filter(grade => grade.id !== id);
    this.render();
  }

  private render(): void {
    this.displayGrades();
    this.displayResult();
  }
```

5. displayGrades: update the grade list

```
private displayGrades(): void {
  this.gradesList.replaceChildren();
  if (this.grades.length === 0) {
    const empty = document.createElement('p');
    empty.textContent = 'No grades yet. Add one above!';
    this.gradesList.append(empty);
    return;
  }

  this.grades.forEach((grade) => {
    const item = document.createElement('div');
    item.className = 'grade-item';
    const info = document.createElement('div');
    info.className = 'grade-info';
    info.textContent = `${grade.name}: ${grade.score.toFixed(1)}% ` +
      `(Weight: ${grade.weight.toFixed(1)}%)`;
```

```
      const button = document.createElement('button');
      button.className = 'delete';
      button.dataset.gradeId = String(grade.id);
      button.textContent = 'Delete';
      item.append(info, button);
      this.gradesList.append(item);
    });
  }
```

We use `textContent` (not `innerHTML`), so user input is never interpreted as HTML.

6. displayResult: show the final grade

```
  private displayResult(): void {
    this.resultDiv.replaceChildren();
    if (this.grades.length === 0) {
      this.resultDiv.textContent = 'Add grades to see your final grade';
      return;
    }

    const summary = calculateFinalGrade(this.grades);  // pure function
    if (summary === null) {
      this.resultDiv.textContent = 'Total weight must be greater than 0';
      return;
    }
```

```
    const letterGrade = getLetterGrade(summary.percentage);
    const gradeText = document.createElement('div');
    gradeText.style.color = GRADE_COLORS[letterGrade];
    gradeText.textContent =
      `Final Grade: ${summary.percentage.toFixed(2)}% (${letterGrade})`;
    const weightText = document.createElement('div');
    weightText.textContent =
      `Total weight: ${summary.totalWeight.toFixed(1)}%`;
    this.resultDiv.append(gradeText, weightText);
  }
}
```

Start the application

```
// Initialize when page loads
document.addEventListener('DOMContentLoaded', () => {
  new GradeCalculator();
});
```

Try It: Let the Compiler Find Bugs

Make one change at a time in `src/app.ts`, run `tsc --project src/tsconfig.json`, and read the error.

1. Typo: `grade.scor` instead of `grade.score`
2. Wrong type: `const grade: Grade = { id: 1, name: 'HW', score: '100', weight: 50 }`
3. Missing field: remove `weight` from the `Grade` object
4. Readonly: add `grade.id = 5;` after creating a grade
5. Remove the `F` entry from `GRADE_COLORS`

In plain JavaScript, every one of these would be found only when a user clicks the button.

What TypeScript Gave Us

- Typos, wrong types, and missing fields are caught before the program runs.
- `Record` and a literal type guarantee that every letter grade has a color.
- `GradeSummary | null` forces us to handle the "cannot calculate" case.
- `calculateFinalGrade` is a pure function: it can be tested without a browser.

Judge It with the Same Questions

1. **Test without a browser?** Yes; `calculateFinalGrade` is pure.
2. **Change the UI independently?** Partly; the class still manages the DOM.
3. **Single source of truth?** Partly; `grades` holds the data, but each change must call `render()`.

What Is Still Left?

- TypeScript protects us from **type** mistakes.
- It does not solve how the **UI is kept in sync** with the state.
- The class still touches the DOM directly.

Next: the **React example**, where the UI is a function of the state.